

20 Perguntas Mais Prováveis da Defesa

Simulador TR1

Perguntas ranqueadas por probabilidade

- Rank 1.** Como vocês modulam com ruído?
- Rank 2.** Como foi feita a modulação no projeto?
- Rank 3.** O ruído entra antes ou depois da modulação? Em qual arquivo isso acontece?
- Rank 4.** Qual é o jeito correto de colocar o bit de paridade? Ele fica no final de cada byte?
- Rank 5.** Por que usar bit de paridade se o CRC-32 detecta melhor?
- Rank 6.** Qual é a vantagem do Manchester em relação ao NRZ-Polar e ao Bipolar AMI?
- Rank 7.** Quais são as vantagens e desvantagens de NRZ-Polar e 16-QAM?
- Rank 8.** Explique o Hamming(8,4) implementado no trabalho.
- Rank 9.** Por que o Hamming vem depois do EDC no transmissor e antes da verificação do EDC no receptor?
- Rank 10.** Como vocês testaram o projeto e como provaram que o CRC-32 está correto?
- Rank 11.** Como funciona o enquadramento com flags, byte stuffing e bit stuffing?
- Rank 12.** Qual é a diferença entre paridade, checksum e CRC-32?
- Rank 13.** Por que 16-QAM é mais sensível ao ruído do que QPSK?
- Rank 14.** O que significa constelação I/Q e para que serve a função de correlação?
- Rank 15.** Como ASK, FSK, QPSK e 16-QAM funcionam especificamente no código?
- Rank 16.** O que acontece se o ruído aumentar muito?
- Rank 17.** Por que o código completa bits com zero em QPSK e 16-QAM? Isso altera a mensagem?
- Rank 18.** Como o receptor decide se recebeu 0 ou 1 nas modulações banda-base?
- Rank 19.** Por que o projeto separa camada física, meio de comunicação e camada de enlace em arquivos diferentes?
- Rank 20.** Se o professor pedir para abrir o código, quais funções você deve saber explicar primeiro?

Gabarito e explicações

Rank 1. Como vocês modulam com ruído?

A resposta correta é: **não modulamos o ruído**. O projeto modula os bits, gera um sinal limpo e só depois o meio de comunicação adiciona ruído gaussiano ao sinal.

Frase pronta:

Nós não modulamos com ruído. Primeiro a camada física gera o sinal limpo a partir dos bits. Depois o meio de comunicação soma ruído gaussiano em cada amostra. O receptor demodula o sinal já ruidoso.

Rank 2. Como foi feita a modulação no projeto?

Existem dois grupos. Em banda-base, os bits viram níveis ou transições de tensão: NRZ-Polar, Manchester e Bipolar AMI. Em portadora, os bits escolhem propriedades de uma onda senoidal: ASK muda amplitude, FSK muda frequência, QPSK usa 2 bits por símbolo e 16-QAM usa 4 bits por símbolo.

```
amostras.append(i * math.cos(angulo) - q * math.sin(angulo))
```

Rank 3. O ruído entra antes ou depois da modulação? Em qual arquivo isso acontece?

O ruído entra **depois** da modulação. A ordem é:

bits -> modulação -> sinal limpo -> meio ruidoso -> sinal recebido -> demodulação

Isso acontece no `meio_comunicacao.py`, não dentro da camada física.

```
ruído = random.gauss(media, sigma)
saida.append(amostra + ruído)
```

Rank 4. Qual é o jeito correto de colocar o bit de paridade? Ele fica no final de cada byte?

Não. No trabalho, a paridade é calculada sobre o **quadro inteiro**, não no final de cada byte. O código soma todos os bits do bloco e calcula a paridade par. Como o simulador trabalha o EDC em bytes, a paridade ocupa 1 byte: sete zeros e o bit de paridade no final.

```
paridade = sum(bits) % 2
return bits + [0] * 7 + [paridade]
```

Frase pronta:

A paridade é por quadro. No nosso código ela ocupa um byte de EDC, mas só o último bit desse byte é a paridade.

Rank 5. Por que usar bit de paridade se o CRC-32 detecta melhor?

Porque a paridade é simples e tem menor overhead. O CRC-32 é mais forte, principalmente para erros em rajada, mas adiciona 32 bits por quadro. A paridade, conceitualmente, precisa de 1 bit; no simulador, ocupa 1 byte por alinhamento do EDC.

Paridade é economia e simplicidade. CRC-32 é robustez. A escolha depende do custo aceito em bits extras e do nível de detecção desejado.

Rank 6. Qual é a vantagem do Manchester em relação ao NRZ-Polar e ao Bipolar AMI?

Manchester sempre tem uma transição no meio do bit. Essa transição ajuda no sincronismo, porque o receptor tem uma mudança previsível dentro de cada intervalo de bit. No projeto, bit 0 é baixo para alto e bit 1 é alto para baixo.

A vantagem é sincronismo: o receptor não depende de longas sequências paradas, porque todo bit tem transição no meio.

Rank 7. Quais são as vantagens e desvantagens de NRZ-Polar e 16-QAM?

NRZ-Polar é simples: bit 1 vira tensão positiva e bit 0 vira tensão negativa. É fácil de implementar e tende a ser mais robusto, mas carrega só 1 bit por símbolo e pode ter longas sequências sem transição.

16-QAM é mais eficiente: carrega 4 bits por símbolo em uma grade I/Q. A desvantagem é que os pontos ficam mais próximos, então o ruído pode deslocar o símbolo para perto do ponto errado.

Rank 8. Explique o Hamming(8,4) implementado no trabalho.

O Hamming(8,4) pega 4 bits de dados e gera 8 bits transmitidos. Ele usa três paridades de Hamming e uma paridade geral:

p1 p2 d1 p4 d2 d3 d4 p0

A síndrome indica a posição de erro simples. A paridade geral permite diferenciar erro simples de erro duplo.

```
p1 = (d1 + d2 + d4) % 2
p2 = (d1 + d3 + d4) % 2
p4 = (d2 + d3 + d4) % 2
bloco = [p1, p2, d1, p4, d2, d3, d4]
p0 = sum(bloco) % 2
```

Rank 9. Por que o Hamming vem depois do EDC no transmissor e antes da verificação do EDC no receptor?

No transmissor, o EDC entra primeiro. Depois o Hamming protege tanto o payload quanto os bits de detecção. No receptor, o caminho é inverso: primeiro tenta corrigir com Hamming, depois verifica o EDC.

```
bloco = ADICIONAR_EDC[config["deteccao"]](bloco)
if config["correcao"] == "hamming":
    bloco = codificar_hamming(bloco)
```

Rank 10. Como vocês testaram o projeto e como provaram que o CRC-32 está correto?

O arquivo `testes.py` testa conversão texto/bits, enquadramentos, paridade, checksum, CRC, Hamming, modulações banda-base, modulações por portadora, ruído e simulação completa.

O CRC-32 é validado com o vetor clássico:

123456789 -> 0xCBF43926

Rank 11. Como funciona o enquadramento com flags, byte stuffing e bit stuffing?

No byte stuffing, o quadro começa e termina com 0x7E. Se 0x7E ou 0x7D aparecem dentro do payload, o transmissor insere 0x7D antes para marcar que aquilo é dado, não controle.

No bit stuffing, a flag é 01111110. Para essa flag não aparecer dentro do payload, o transmissor insere um zero depois de cinco bits 1 consecutivos.

Rank 12. Qual é a diferença entre paridade, checksum e CRC-32?

Paridade é simples e fraca: detecta quantidade ímpar de erros. Checksum soma palavras de 16 bits em complemento de 1 e é intermediário. CRC-32 usa divisão polinomial em base 2 e é o detector mais forte do projeto.

Robustez: paridade < checksum < CRC-32. Custo no simulador: paridade 1 byte, checksum 2 bytes, CRC-32 4 bytes.

Rank 13. Por que 16-QAM é mais sensível ao ruído do que QPSK?

Porque 16-QAM tem 16 pontos na constelação, enquanto QPSK tem 4. Como há mais pontos no mesmo espaço de sinal, eles ficam mais próximos. Com ruído, o ponto recebido pode ser deslocado e o receptor pode escolher o vizinho errado.

16-QAM ganha eficiência, mas perde margem contra ruído.

Rank 14. O que significa constelação I/Q e para que serve a função de correlação?

Constelação I/Q é representar cada símbolo por duas coordenadas: I, em fase com o cosseno, e Q, em quadratura com o seno. Na transmissão, o código gera a onda usando essas coordenadas. Na recepção, a correlação estima qual par I/Q estava no sinal recebido.

```
soma_cos += amostra * math.cos(angulo)
soma_sen += amostra * math.sin(angulo)
i = 2 / N * soma_cos
q = -2 / N * soma_sen
```

Rank 15. Como ASK, FSK, QPSK e 16-QAM funcionam especificamente no código?

ASK muda amplitude: bit 1 transmite portadora, bit 0 transmite amplitude zero. FSK muda frequência: bit 0 usa uma frequência e bit 1 usa outra. QPSK agrupa 2 bits por símbolo e escolhe um ponto I/Q. 16-QAM agrupa 4 bits por símbolo: 2 bits escolhem I e 2 bits escolhem Q.

ASK e FSK carregam 1 bit por símbolo. QPSK carrega 2. 16-QAM carrega 4.

Rank 16. O que acontece se o ruído aumentar muito?

O demodulador começa a errar bits. O Hamming pode corrigir erro simples por bloco, mas erros múltiplos podem ser apenas detectados ou até escapar, dependendo do EDC usado.

Quanto maior o ruído, maior a chance de o sinal recebido cair do lado errado da decisão.

Rank 17. Por que o código completa bits com zero em QPSK e 16-QAM? Isso altera a mensagem?

QPSK precisa de grupos de 2 bits e 16-QAM precisa de grupos de 4 bits. Se o último grupo não fecha, o código completa com zeros só para formar o último símbolo. Isso não altera a mensagem útil, porque a camada de enlace e a aplicação recuperam apenas os bits pertencentes aos quadros válidos.

```
def pad(bits, tamanho):
    resto = len(bits) % tamanho
    if resto == 0:
        return bits
    faltam = tamanho - resto
    return bits + [0] * faltam
```

Rank 18. Como o receptor decide se recebeu 0 ou 1 nas modulações banda-base?

No NRZ-Polar, o receptor calcula a média das amostras do bit: média positiva vira 1, caso contrário vira 0. No Manchester, compara a média da primeira metade com a segunda: se começou mais alto e terminou mais baixo, é 1; se começou baixo e terminou alto, é 0. No Bipolar AMI, olha o módulo da média: se há pulso, é 1; se está perto de zero, é 0.

Rank 19. Por que o projeto separa camada física, meio de comunicação e camada de enlace em arquivos diferentes?

Porque cada arquivo representa uma responsabilidade da pilha. A camada de enlace cria quadros, adiciona detecção e correção. A camada física transforma bits em sinal e sinal em bits. O meio de comunicação simula o canal, adicionando ruído. Essa separação deixa o código mais fiel ao modelo de redes e facilita explicar onde cada fenômeno acontece.

Rank 20. Se o professor pedir para abrir o código, quais funções você deve saber explicar primeiro?

As funções mais importantes são:

- `meio_comunicacao.transmitir`: mostra onde o ruído entra;
- `camada_fisica.onda` e `correlacionar`: explicam portadora e I/Q;
- `modular_manchester`: explica sincronismo;
- `adicionar_paridade_par`: explica paridade por quadro;
- `calcular_crc32`: explica CRC manual;
- `codificar_hamming` e `decodificar_hamming`: explicam correção;
- `transmitir` e `receber` da camada de enlace: explicam a ordem do pipeline.